

Transformer, Large Language Model, Retrieval-Augmented Generation and Multi-modal systems

Prof. Diego Klabjan, Dr. Raman Khurana

Center for Deep Learning

Northwestern University

Introduction

Artificial Intelligence (AI) has seen transformative advancements over the past few years, with several groundbreaking technologies shaping the landscape of machine learning and deep learning. This report describes some of the most significant innovations, including **Transformers, Large Language Models (LLMs), Retrieval-Augmented Generation (RAG),**

and **Multimodal AI**. These technologies have revolutionized how we approach natural language processing (NLP), information retrieval, and the integration of diverse data modalities to solve complex problems.

The scope of this document is multifold. It starts with the architecture of Transformer model and how it is used for Large Language Models (LLMs). It discusses in detail some use cases that can help business automate manual and human intensive tasks using LLMs and how they can be used with internal data of the business without data leaving the premise, method known as Retrieval Augmentation Generation (RAG). The report also discusses various methods that can be used improving the basic RAG pipeline and close the report with introduction and applications of Multi-modal AI within businesses.

First, it explores the evolution of Transformers, from their introduction in 2017 through the paper "*Attention Is All You Need*" to their dominance in sequence modeling tasks.

Transformers have addressed the inefficiencies of earlier architectures like RNNs and LSTMs by leveraging self-attention mechanisms, enabling superior scalability, parallelism, and contextual understanding. This document introduces the foundational principles of the Transformer architecture, including attention mechanisms, positional encodings, and encoder-decoder frameworks. It also highlights how these elements collectively form the backbone of state-of-the-art models in NLP and generative tasks.

Second, the discussion shifts to **Large Language Models (LLMs)**, which have emerged as a powerful application of Transformers. Trained on massive datasets across diverse domains, LLMs like GPT, Llama, and Claude have demonstrated remarkable capabilities in text generation, summarization, and multilingual translation. The document categorizes LLMs by attributes such as size, accessibility (open vs. closed source), and suitability for specific use cases. It also outlines their utility in real-world applications, from customer assistance and document summarization to fine-tuned predictive tasks like time-series forecasting. This section also discusses the tools and libraries, such as LangChain, Hugging Face, and LlamaIndex, that facilitate the technical implementation of LLMs and their integration into scalable applications.

Third, the document examines **Retrieval-Augmented Generation (RAG)**, a technique designed to bridge the limitations of LLMs in handling domain-specific or real-time data. RAG combines LLMs with external knowledge bases, enabling accurate and context-aware responses by dynamically retrieving relevant information. This section provides an overview of the RAG pipeline, including indexing, retrieval, augmentation, and generation, and discusses its critical role in applications like domain-specific Q&A, customer support, and content generation. It also introduces advanced techniques, such as query translation

and query fusion, that further enhance the performance of RAG systems. Specific tools like FAISS, Pinecone, and Elasticsearch are highlighted for their roles in building efficient RAG pipelines.

Finally, the document addresses the emergence of **Multimodal AI**, which extends beyond the textual capabilities of LLMs by integrating data from multiple modalities, such as text, images, audio, and video. By emulating the human ability to process diverse sensory inputs, multimodal AI systems achieve a comprehensive understanding of data, enabling applications like sentiment analysis, multimedia content interpretation, and human-computer interaction. The document outlines the principles of heterogeneity, connections, and interactions that underpin multimodal AI, emphasizing its potential to revolutionize fields where multimodal data is critical. Additionally, the tools and frameworks enabling multimodal AI, such as CLIP, DALL-E, and Whisper, are discussed for their technical implementations and applications.

This report not only focuses on the concepts underlying these transformative technologies but also highlights the tools and libraries that enable their practical implementation. Furthermore, it discusses a diverse set of use cases that can serve as potential projects or applications for companies, helping industry practitioners leverage advancements in the Generative AI domain. It serves as a guide for practitioners and industry professionals to address complex, real-world challenges while capitalizing on the opportunities these technologies present.

Transformers

History

Before the release of Transformer in 2017 with the publication of the paper "Attention Is All You Need,"[1] sequence modeling was dominated by Recurrent Neural Networks (RNNs) [2,3], Long Short-Term Memory (LSTM) [4] networks and their variants. These models were effective for tasks like language modeling and translation, but they encountered

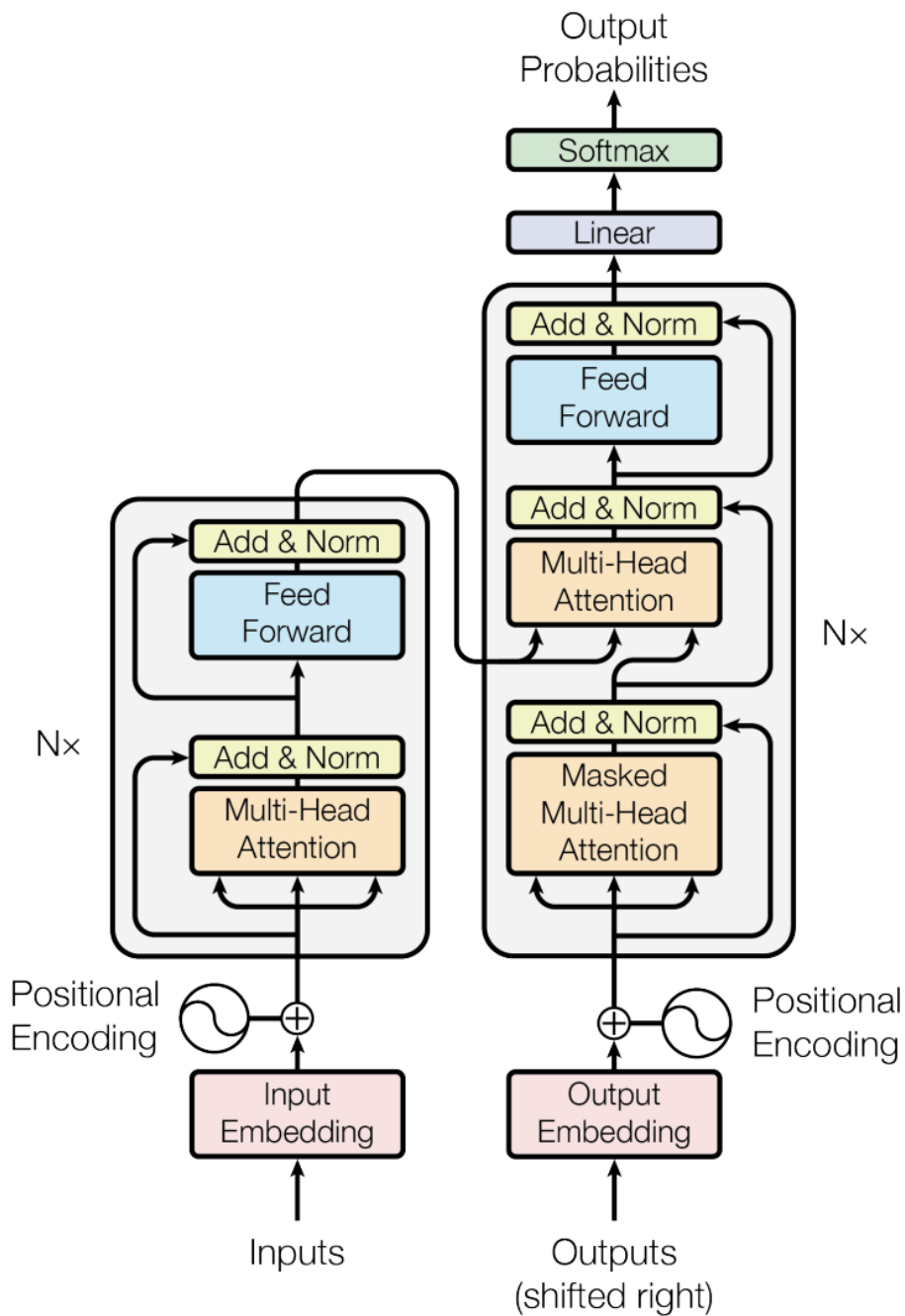
challenges when scaling large datasets essential for generative tasks, such as conversational AI [5] and multilingual machine translation [6].

RNNs and LSTMs process data sequentially, relying on the hidden state to carry information from previous steps and update it at each timestep. While this design enables capturing temporal dependencies crucial for language processing, it limits scalability and slows down the computation. Furthermore, the sequential processing creates a dependency chain that hinders parallelization, leading to inefficiencies in handling long sequences.

In addition to these limitations, these architectures struggle with learning long-range dependencies due to vanishing gradient issues, where gradients diminish as they propagate through multiple layers. For instance, earlier layers in deep networks often fail to receive sufficient updates, resulting in suboptimal learning of deeper context. These challenges made traditional approaches less effective for large-scale and complex sequence modeling tasks.

Efforts to address these shortcomings included gated mechanisms and hybrid architectures that combined RNNs with attention [7,8]. However, these solutions still relied heavily on sequential operations, which bottlenecked training speed and scalability. Convolutional Neural Networks (CNNs) were another alternative explored for sequence transduction tasks [9]. CNNs, as used in models like ByteNet [10] and ConvS2S [9], reduced sequential computations by processing all sequence positions in parallel. However, they struggled to model dependencies between distant positions efficiently, as the number of layers required to connect distant positions increased logarithmically or linearly with sequence length. This led to an inherent limitation in capturing global context efficiently in long sequences.

The Transformer architecture emerged as a solution to these challenges by eliminating recurrence and convolution entirely, instead leveraging attention mechanisms to compute global dependencies across sequences more effectively and efficiently.



Introduction to Transformer

The Transformer architecture was introduced in the paper "Attention Is All You Need" by Vaswani et al.. This paper marked a breakthrough in sequence-to-sequence models by replacing recurrence and convolutional layers with self-attention mechanisms. Unlike traditional encoder-decoder models that rely on RNNs or CNNs, the Transformer relies

solely on attention mechanisms to model dependencies across sequences. This architecture (Figure above, from paper [1]) achieves high efficiency and parallelizes the training process.

Encoder-Decoder Framework

At the core of the Transformer lies an encoder-decoder structure. The input flows through the encoder, which processes it into a series of intermediate representations known as the encoder states. These states capture contextual information about the input sequence, allowing the model to understand relationships between different parts of the data. The encoder's output is then passed to the decoder, which uses these intermediate states, along with a shifted version of the target sequence, to generate predictions step by step. Finally, the decoder outputs probabilities for each position in the target sequence, representing the likelihood of each possible token in the vocabulary. This architecture enables the Transformer to handle complex sequence-to-sequence tasks efficiently, such as machine translation or text generation

Each encoder and decoder are composed of many components as marked in the architecture diagram. The report explains each one of them before explaining the full functionality of the model. In sequence data e.g. a sentence is first converted into tokens via Tokenization¹. Tokenization is a prerequisite for the Transformer's input processing. These tokens are then mapped into continuous representations, which are numerical vectors holding the meaning of the input tokens. For example, a given word in a sentence is represented not just as a discrete token but as a high-dimensional vector² that captures semantic and syntactic properties. The decoder then takes these continuous representations and generates output sequences. It uses the information encoded by the encoder to make predictions about the next tokens in the sequence, ensuring coherence and relevance.

Both encoder and decoder stacks consist of six identical layers, each comprising a multi-head self-attention mechanism and a position-wise feed-forward network. Each layer in the encoder contained a multi-head self-attention mechanism and a position-wise fully connected feed-forward network. The self-attention mechanism enables the encoder to focus on different parts of the input sequence simultaneously, capturing complex relationships between tokens (see next section for more details). The decoder part of the

¹ Tokenization is the process of breaking down the sequence of text into small units such words, subwords or characters.

² This is also called dense embedding of the input tokens.

architecture is similar to that of encoder but includes a third component i.e. masked multi-head attention. Decoder also uses a masked multi-head attention sub-layer instead of multi-head attention as in encoder. Masking in the decoder's self-attention ensures predictions for a position depend only on previously generated tokens and not the tokens in the future³. In addition, the multi-head attention in decoder uses the encoder's output and shifted output. This ensures the decoder can effectively incorporate the encoded information when generating outputs.

As clear from the discussion so far, Transformer architecture introduces multiple new concepts which makes it different from existing models and these new concepts put together makes Transformer architecture revolutionary. Let's focus on four important parts of the architecture with an example:

1. Input Embeddings
2. Positional encoding
3. Attention mechanisms
4. Feed Forward network

Input Embeddings and Positional Encoding

Transformers were developed for language translation and found application in wide landscape. Let's see how a sentence "I love France!" can be converted to French using Transformer Architecture. Input sentences are first converted into tokens, which are the smallest units of text processed by the model. Tokenization is the process of splitting a sentence into these smaller parts, which can be words, subwords, or even characters, depending on the model's design. The token format of input might look like ["I", "love", "Fr", "ance", "!",], where the model breaks down the text into tokens. For example, 'France' is split into 'Fr' and 'ance' to align with tokenization methods like those used in GPT-3. This splitting helps the model handle rare or unknown words effectively. This is in line with GPT 3 model that uses 1-4 characters for a token. Two special tokens start of sequence (SOS) and end of sequence (EOS) are added to the list of tokens. For simplicity, we can use [<SOS>, "I", "love", "France", "!", <EOS>]. Each token in the list is then converted into a dense embedding, which is a continuous numerical representation derived from the model's vocabulary. This embedding captures the semantic and syntactic properties of each token, enabling the model to interpret and process the input effectively. For transformer model each token is converted to a vector of dimension 512. The dense

³ This autoregressive behavior of masked multi-head attention is highly desirable to ensure there is no data leakage and predictions are made only using past data.

embedding (input embedding as in the architecture) has information about the words/tokens in the input sequence but it does not contain any information about their relative position in the sequence. This relative position information is combined with the embedding using positional encoding. Positional encoding ensures that the model understands the order of tokens, which is crucial for preserving the structure and meaning of the input sequence. The relative position of a token w.r.t the sequence and embedding vector is then converted into vector of same size as dense embedding. The sum of dense embedding and positional encoding is then served as input to the encoder block of the Transformer.

Attention Mechanisms

The Transformer's multi-head attention mechanism is the main reason for its success, enabling it to compute dependencies between tokens regardless of their distance in the input sequence. This ability is often described as capturing the relationship between tokens, as it allows the model to weigh the importance of each word relative to others in the sequence. By doing so, attention mechanisms identify and emphasize meaningful relationships, such as how words in a question relate to an answer or how adjectives describe specific nouns. For example, in the sentence "I love France!", the attention mechanism may prioritize the connection between "I" and "love" to establish the subject-verb relationship and then "love" and "France" to infer the object of affection. The capability to interpret context and meaning across varying distances in the sequence underpins the Transformer's effectiveness in language tasks. The multi-head attention is based on the concept of self-attention. So, let's understand self-attention before moving to multi-head attention.

Self-Attention

Self-Attention⁴ allows the model to evaluate the importance of each word with respect to all other words in the input sequence. Imagine processing the sentence "I love France!". To understand its meaning, it is essential to establish relationships between the subject ("I"), the verb ("love"), and the object ("France"). The embedding vector obtained after position encoding is fed used estimation of the strength of interaction of tokens in the sequence.

⁴ Also called scaled dot-product Attention

This mechanism works by generating three types of vectors for each token in the sequence, Query, Key and Value.

- **Query:** Represents the current word being focused on.
- **Key:** Represents the potential connections to other words in the sequence.
- **Value:** Contains the information about the token that could be relevant to other tokens.

These are obtained by performing linear transformation of the embedding vector in different space. The terminology of Query, Key and Value is borrowed from database or python like dictionary. Each query is compared with all keys to compute the attention scores (like searching for match in database). The corresponding values weighted by attention score are then used to compute the output.

When processing the token “love”, its query vector interacts with the key vectors of all tokens in the sequence (“I”, “love”, “France”, and “!”). This interaction is measured by calculating the dot product between the query vector and each key vector, yielding attention scores. These scores indicate the strength of the relationship between “love” and the other tokens.

To ensure numerical stability and balance, these scores are scaled down and passed through a softmax function. The softmax normalizes the scores into probabilities that sum to one. These probabilities are then used to weight the value vectors of all tokens, resulting in a weighted sum that provides the context-rich representation for “love”. This representation encapsulates how “love” relates to the entire sentence, dynamically highlighting relevant parts.

This process, known as self-attention, enables the model to “attend” to significant parts of the input, making decisions based on their importance within the context of the task. Self-attention is particularly valuable for capturing both local and global relationships across the sequence.

Multi-Head Attention

The self-attention as described in previous section provides a single perspective on token relationships. Multi-Head Attention enhances this by allowing the model to consider multiple perspectives simultaneously. Each “head” in Multi-Head Attention independently projects queries, keys, and values into smaller subspaces. This enables the model to capture different types of relationships within the sequence.

For example, in “I love France!”, one attention head might focus on the subject-verb relationship (“I” and “love”), while another examines the sentiment expressed between

“love” and “France”. A third head could even focus on punctuation (“France” and “!”), recognizing its role in emphasizing the sentiment.

As a fun fact, Transformer paper used 8 attention heads to model the token relationship and GPT 3 used 96 attention heads. Each head processes a subset of the input features independently (in parallel), allowing the model to learn diverse relationships within the data.

After processing, the outputs of all attention heads are concatenated and linearly transformed into a single vector. This comprehensive vector combines insights from multiple relational viewpoints, enriching the representation of the token. Multi-Head Attention thus provides a good understanding of relationship between tokens in the input sequence, enabling the Transformer to handle complex language tasks with remarkable effectiveness.

Masked Multi-Head Attention

Masked Multi-Head Attention is a specialized version of Multi-Head Attention used in the Transformer’s decoder. Its primary purpose is to preserve the autoregressive⁵ property during sequence generation. In autoregressive tasks like text generation, each token is predicted based only on previous tokens, not on future ones.

To enforce this behavior, Masked Multi-Head Attention employs a masking mechanism. This mechanism prevents any token from attending to tokens that appear later in the sequence. For instance, when generating the sentence “I love France!”, the model ensures that while predicting “love,” it attends only to “I” and not to “France” or “!”. Similarly, when predicting “France,” it attends to both “I” and “love” but not to “!”.

The implementation involves setting the attention weights for future tokens to negative infinity before applying the softmax function. This effectively zeroes out these weights, ensuring the model cannot consider information from future tokens.

By incorporating this masking, the Transformer ensures that the decoder generates sequences incrementally and maintains coherence with previously generated tokens. Masked Multi-Head Attention is thus crucial for tasks like machine translation and text completion, where the output sequence must be generated step by step while retaining context from prior outputs.

⁵ Next value or prediction will depend on the past values.

Training and Inference in Transformers

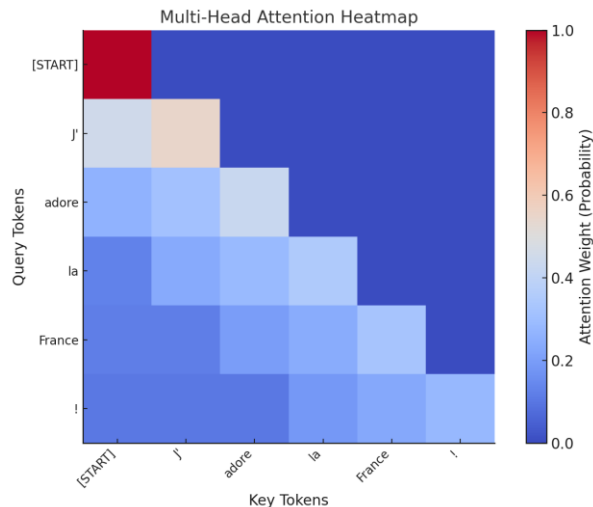
The Transformer operates differently during training and inference, optimizing its design to handle entire sequences efficiently during training while generating outputs step-by-step during inference. Now that we have all the ingredients needed to understand the Transformer let's see how this architecture is trained and how to make inference using trained model.

Training

During training, the Transformer processes both the input and target output sequences simultaneously. For instance, in a translation task where the input is "I love France!" and the target output is its French equivalent, "J'adore la France !", the entire input sentence is passed through the encoder, while the shifted version⁶ of the target output sequence (e.g., "[<SOS>, J', adore, la, France, !]") is fed into the decoder.

The encoder generates continuous representations for the input sentence by embedding and processing all tokens simultaneously. These continuous representations capture the semantic and syntactic relationships within the input sequence. Similarly, the decoder processes the entire shifted target sequence simultaneously during training, where the masked multi-head self-attention ensures that each token only attends to its preceding tokens (and not future tokens) to maintain the autoregressive property. For example, while processing the shifted target sequence "[<SOS>, J', adore, la, France, !]", the masked multi head attention builds the relationships within the target sequence by focusing only on the prior tokens. Figure below shows an illustration of the masked multi-head attention, presenting the how much attention a given token gives to other tokens in the sequence.

⁶ The decoder is designed to predict the next token autoregressively. It uses the output from encoder and previously predicted tokens from decoder to make the prediction for next token. By shifting, decoder added a start token at the beginning of the sequence and ensures the decoder sees only tokens that precede the current position.



Note that there are two attention blocks in the decoder part of the architecture. The masked attention block learns how each token of output sequence should attend to preceding tokens in the output sequence, the multi-head attention block in the decoder is called a cross-attention or encoder-decoder attention. This multi-head attention block allows the decoder to use the contextual information encoded by the encoder and combine it with the decoder's own input. The query (Q) vectors come from the decoder (i.e. output of decoder masked attention block) and the key (K); value (V) vectors come from the encoder. This block computes how much attention the decoder should pay to each token in input sequence (encoded by the encoder) for the current query token. For instance, when processing the query token "adore," the decoder aligns its attention to the encoded representation of the input sequence, "I love France!". The encoder provides the contextualized representations (keys and values) for the input tokens. The cross-attention block computes how much attention "adore" should pay to tokens like "I" or "love" in the input, emphasizing relevant information like "love" due to its semantic alignment with "adore". Meanwhile, the decoder ensures it does not access future tokens in the target sequence, such as "la" or "France," due to the autoregressive property enforced by the masked attention block.

The model's objective during training is to minimize the loss function, typically the **cross-entropy loss**, which quantifies the difference between the predicted probabilities and the true labels (target tokens). For each token in the output sequence, the cross-entropy loss penalizes predictions that diverge from the ground truth, guiding the model to improve over successive iterations.

By attending to all tokens in the input sequence simultaneously and leveraging masked multi-head attention, the model effectively captures complex relationships and trains efficiently on large datasets.

Inference

During inference, the Transformer generates the output sequence one token at a time. For example, consider translating the English sentence “I love chocolate!” into French. The input sequence is processed by the encoder to produce a fixed set of continuous representations.

The decoder starts with the start-of-sequence token “[]” and predicts the first token. If the model is trained on sufficiently large dataset and it has learned from training data, then it should predict the first token as “J’ ”. As a next step of inference, model now have “J’ ” as input to predict the next token, i.e. “adore,” and continues this process iteratively. At each step, the decoder uses Masked Multi-Head Attention to attend to both the encoder’s outputs and its own previously generated tokens while ensuring the autoregressive property. The masking mechanism blocks future tokens, ensuring that the generation of each token relies only on past context.

Note that during the training process the model processes the whole sentence in a single time stamp but during inference the tokens are predicted sequentially. This step-by-step generation ensures the coherence of the output sequence. Although inference is inherently sequential, it allows the Transformer to produce accurate translations or other sequential outputs based on the learned relationships in the data.

Large Language Models (LLMs)

The backbone of a Large Language Model (LLM) is typically a Transformer architecture that can capture complex patterns and context from the data during the training process. LLM

is a type of AI model that is trained on vast amount of text data to understand and generate human like language response. LLMs are capable of learning complex patterns in the textual data during the training process and generalize the learning to new text they receive for applications like text summarization, question-answering and language translation etc. These models typically have billions of parameters and require significantly huge computing resources for training. Due to the huge computational requirement the training cost of LLMs is in the order of Millions/Billions of Dollars.

The pre-trained LLMs are trained on vast amount of data using almost all domains, e.g. science, geography, coding, arts, history, language etc. During the training process LLMs learn patterns and semantic relationship in the input data and save this learned knowledge in the form of (typically billions of) training weights. One can also think of it as compressing all the knowledge in input text into these weights. This enables them to answer any question asked from a very diverse domain.

Characteristics of LLMs

LLMs are complex architectures and have many unique characteristics, a few of them⁷ are its architecture, training data, task completion, parameter scaling. These are briefly discussed below:

Architecture

As mentioned, LLMs use Transformer architecture (or its variant) to learn the relationship between input sequence data used for training. How LLMs are trained or discussion on the full architecture of LLM is beyond the scope of this report. But it is worth nothing that most LLMs are simply the transformer architecture at very large scale⁸ and trained using huge text dataset. LLMs like GPT (Generative Pre-trained Transformer) and BERT (Bidirectional Encoder Representations from Transformers) are built on variations of this architecture. GPT models use a **decoder-only** architecture, making them highly effective for generative tasks, while BERT employs an **encoder-only** architecture, excelling in understanding and classification tasks.

⁷ this is not an exhaustive list.

⁸ by very large scale we mean, the number of encoder/decoder units, number of heads in multi-head attention blocks, hidden layer and number of hidden neurons in the Feed Forward Network and resulting number of trainable parameters.

Training Data

The performance of LLMs is heavily dependent on the quality and quantity of their training data. These models are typically trained on massive datasets comprising diverse text sources, including books, articles, websites, and social media. LLMs are trained on terabytes of text data. For example, GPT-3 was trained on approximately 570GB of text from sources like Common Crawl, Wikipedia, and books. The data used for training covers a wide range of topics, languages, and styles, enabling the model to generalize across domains. Text data is tokenized and cleaned to remove noise, ensuring high-quality input for training.

Task completion

One of the most striking features of LLMs is their ability to perform a wide range of tasks without task-specific training. This is achieved through pre-training and fine-tuning:

Pre-training

During pre-training, the model learns to predict the next word in a sentence (for autoregressive models like GPT) or to fill in missing words (for masked language models like BERT). This process imbues the model with a general understanding of language.

Fine-tuning

After pre-training, the model can be fine-tuned on specific tasks, such as sentiment analysis, translation, or question answering. Fine-tuning requires significantly less data than training from scratch, making LLMs highly efficient.

Zero-shot One-shot, and few-shot learning

Zero-shot, one-shot, and few-shot learning are paradigms that demonstrate the remarkable adaptability and generalization capabilities of LLM like GPT-3.

In zero-shot learning, the model performs a task without any explicit examples or prior training specific to that task. Instead, it relies solely on its pre-trained knowledge and the instructions provided in the input prompt. For instance, if asked to translate a sentence from English to French, the model can generate the correct translation even if it was never explicitly trained on translation tasks, leveraging its understanding of language patterns and relationships.

One-shot learning takes this a step further by providing the model with a single example of the task within the input prompt. This example serves as a reference, allowing the model to understand the structure of the task or pattern and apply it to new inputs. For example, if given one example of sentiment analysis, such as classifying "I love this movie!" as

"positive," the model can then classify a new sentence like "The food was delicious" as "positive" based on the single reference.

Few-shot learning extends this concept by providing the model with a small number of examples (typically 2 to 5) in the input prompt. These examples help the model better understand the task, enabling it to generate more accurate and contextually appropriate outputs. For instance, if tasked with writing a rhyming poem, the model can use a few provided examples of rhyming poems to generate a new one that adheres to the same structure and style. Together, these paradigms highlight the flexibility and efficiency of LLMs, allowing them to perform a wide range of tasks with minimal or no task-specific training, reducing the need for large, labeled datasets and making them highly versatile tools for various applications.

Parameter scaling

The size of an LLM is often measured by the number of parameters, learnable weights that the model uses to make predictions. Early models like GPT-1 had 117 million parameters, while GPT-3 boasts 175 billion parameters. There are several aspects that change with parameter scaling; e.g., its performance, capabilities and cost. Increasing the number of parameters generally improves the model's performance, as it can capture more complex patterns and nuances in the data. Larger models exhibit diverse abilities, they have capabilities that are not present in smaller models. For example, GPT-3 and 4 can perform arithmetic, translate languages, and even generate creative content, tasks that smaller models struggle with. However, scaling comes with significant costs. Training and deploying large models require massive computational resources.

Landscape of LLMs

One can categorize the LLMs based on many aspects like availability of the model weights, pricing, performance for a specific task, and number of parameters. We categorize them based on two such aspects: availability and scale. This gives an overall idea of how the landscape of LLMs looks.

Private vs Public⁹

⁹ This is also a categorization of Paid vs Free. The private LLMs are generally paid or free with very limited capabilities and public models are free to use.

This categorization comes from the fact that some LLMs (weights) are publicly available for further tuning and improvement for desired use cases (public, or open source) and some are not (private/closed). The closed source LLMs are accessible via API and come at a price of per token usage.

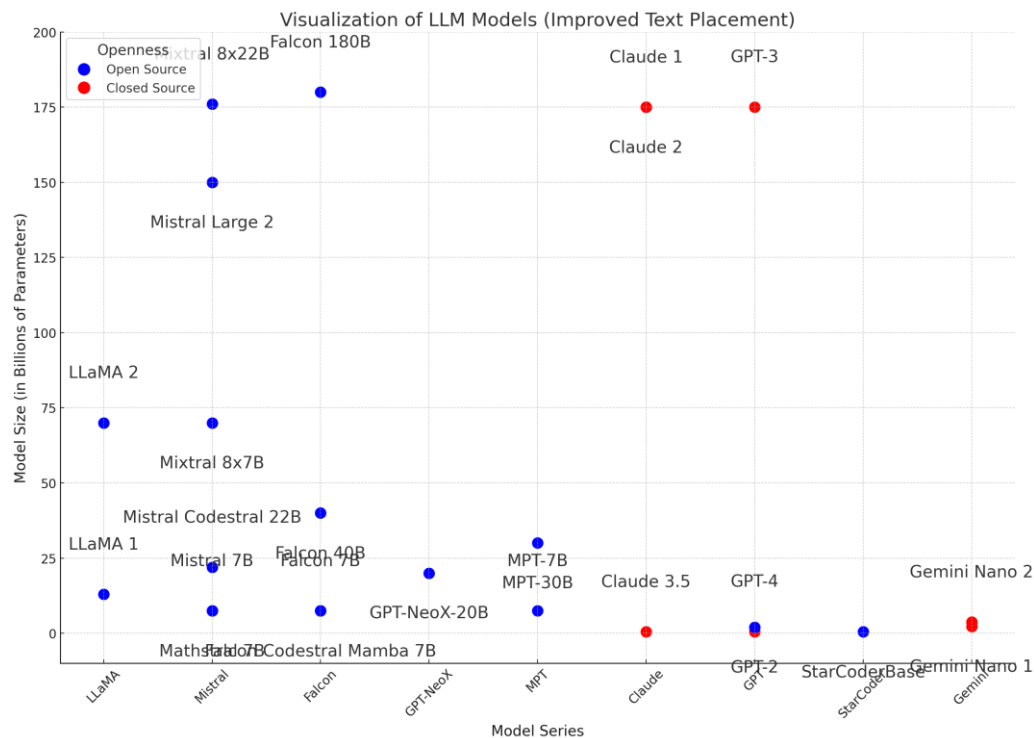
Number of parameters

Another criterion to distinguish between the capabilities is the number of trainable weights. Generally, the higher the number of parameters of an LLM the more knowledge is encapsulated in the model. Due to the huge number of model parameters, they take longer processing time and relatively more storage space and computational resources.

Since the release of ChatGPT there have been release of many LLMs from different organizations and companies. Most of them are trained on diverse datasets and answer a wide variety of questions but some are trained/fine-tuned to solve a specific problem. Following is the list of series that have multiple versions of models based on size and capabilities of the LLMs.

1. Llama by Meta
2. Mistral by MistralAI
3. Falcon by TII
4. GPT series by OpenAI
5. Claude by Anthropic
6. Gemini by Google
7. OpenHermes

Figure below shows a comparison of various Large Language Models available in private and public space in different sizes. In addition to these there are a few super models with >200 B parameters. These are Llama 405 B, Gemini Pro with 600 B, and Gemini 1.0 Ultra with 1500 B parameters.



It is hard to categorize based on all aspects of LLMs so we used only two of them. The snapshot of the table below shows the complete list of LLMs available along with more important parameters to decide which LLM to use for a given use case. The full live list of the table can be accessed at [GitHub](#).

Model Series	Model Version	Number of Parameters	Open Source & License Terms	Features	Date of Release	Context Length	Pricing
LLaMA	LLaMA 2	7B, 13B, 70B	Custom License (research and commercial use with restrictions)	General-purpose language understanding and generation	July 2023	4,096 tokens	Free for approved users
LLaMA	LLaMA 3.1	8B, 70B, 405B	Custom License (research and commercial use with restrictions)	Advanced language tasks, coding, multilingual support	April 2024	Up to 128,000 tokens	Free for approved users
Mistral	Mistral 7B	7.3B	Apache 2.0	General-purpose language tasks; outperforms larger models on various benchmarks	July 2024	128,000 tokens	Free
Mistral	Mixtral 8x7B	Mixture of Experts (MoE) model with 7B active parameters per expert	Apache 2.0	Efficient handling of diverse tasks through MoE architecture	July 2024	128,000 tokens	Free
Mistral	Mistral Large 2	123B	Apache 2.0	High-performance model for complex language tasks	July 2024	128,000 tokens	Free

Some of these LLMs can be accessed via Chatbot and you might have already some experiences with ChatGPT or Claude or Gemini, DeepSeek, etc. This might be productive for personal use but that is not the most appropriate use of the LLMs when it comes to using LLMs to solve business problems. LLMs are capable and well suited for any repetitive task (or even creative) at the business that require response in natural language. The development of tools/libraries to allow the programmatic use of LLMs enables the building of scalable applications to solve business problems.

Toolkit for LLM

As time passes the landscape for LLMs is continuously evolving and so are the tools to access LLMs programmatically. Following is the list of tools/libraries that allow widely used in the academia and industry for a variety of use cases:

1. LangChain
2. Hugging Face Transformers
3. GPT4All
4. LlamaIndex

However, if one is willing to run the LLM model locally and host the open-source model on a local hardware it requires an additional step to host the LLM model. This can be done via one of the following tools depending on the use case and other requirements, e.g. hardware compatibility, model support, documentation and community support, and frequency of updates.

1. Ollama
2. LM Studio
3. GPT4All
4. Jan
5. Llama.cpp

Not all the tools are compatible with all Large Language Models or hardware. So, when running locally, it is important to decide the model and appropriate tool. Running an LLM locally requires some extra effort, but this comes with its own benefits like Privacy and Security, faster response time, cost saving and customizability are a few of those.

Applications of LLMs

Large Language Models perform far exceeding the expectations to perform many generic tasks and hence find many applications in the business. In general, LLMs offer reduced human workload, faster response time with lower operations cost. We list here example use cases and discuss a few of them in detail.

- **Summarization:** As mentioned LLMs perform beyond expectations for many generic language tasks and text summarization is one of them. When prompted correctly, it is both accurate and fast to summarize long documents with the help of

these AI models. There are various techniques that can be used to summarize the documents (language data) using LLMs:

- **Extractive summary:** LLM extracts important/relevant sentences or key phrases from the document. E.g. listing from legal or compliance documents or citation of a relevant document.
- **Abstractive summary:** LLM summarizes the key points in its own words instead of extracting the sentence/phrase from the document.
- **Hybrid summary:** LLM first extracts the key phrases and then summarizes in its own words to avoid exact verbatim of the original document. E.g. Summary of long product review.
- **Query based summary:** LLM answers specific queries or questions about a document. E.g. What are the compliance risks mentioned in this policy document?
- **Multi-document summary:** LLM can access multiple documents and summarize them together in a single concise report.
- **Regulatory Compliance & Document Review:** Financial and insurance businesses face extensive regulatory requirements. LLMs can be used to read lengthy policy documents, regulatory notices, or legal briefs and generate concise, role-specific summaries highlighting key obligations and potential risks. By automating the review and summarization process, companies can maintain compliance more efficiently and reduce legal exposure.
- **Customer Assistance:** LLMs are powerful tools to understand and generate natural language and this makes and apt to automate the highly manual and expensive process of customer assistance for any business. There are several ways LLMs can be deployed for customer assistance and enhance customer experience.
 - **Automated chatbot and virtual assistant:** Automated chatbot have been around for quite some time but chatbots powered with LLMs offer real time response to complex questions, troubleshooting and take care of FAQs. e.g. An insurance company uses LLM powered bot to guide customers through policy benefits to help them choose the right insurance policy. Such a chatbot can provide detailed Information and answer questions by potential customers to compare different policies and hence increasing the conversion rate.
 - **Multilingual support:** LLMs can handle interactions with customers in multiple languages without dedicated teams focused on each language.
 - **Claim Processing Assistance:** LLMs can streamline the insurance claim filling process by guiding customers through documents and status.

- **Human Resource:** Pretrained LLM embeddings can help HRs finding the best candidate based on the job description and candidate resume.
- **Time Series Forecasting:** Although LLMs are pretrained for natural language data but they can be prompted to perform complex predictive tasks like time series forecasting. There have been many efforts in academia and industry to fine tune or train the Language model to learn the complex patterns, trends and seasonality based on huge amount of time series data. This gives LLMs the capabilities to forecast the time series in unseen future.

Retrieval Augmented Generation (RAG)

The knowledge about diverse domains makes LLM a very good source to answer generic questions but they suffer from two major limitations; (1) LLM knowledge is limited to the data it was trained on, they are blind to data created beyond a timestamp or another domain (2) they are often not capable of answering business questions, answer to which is present in private dataset of a company or organization. We discussed about zero-, one- and few-shot learning that can work nice for simple use cases. There are other ways to enhance the capabilities of the LLMs and upskill LLMs for a specific task (using this new data generated in the due course):

- Training a new LLM from scratch with domain specific dataset: Computing, data and budget intensive¹⁰.
- Fine-tuning of the LLMs: Much cheaper than training the LLM but still significantly computationally and budget expensive.
- Retrieval Augmented Generation: Economical on computing and budget.

RAG is one of the most popular ideas to build applications using LLMs today. This section focuses on what RAG is and why it is so powerful to enhance the capabilities of LLMs.

LLMs are trained mostly on public data and most of the data in the world is private. LLMs trained on public data are already capable of answering a wide variety of queries, but modern business problems are not so trivial and require private/proprietary data from businesses to answer complex decision-making questions. Without the ability to incorporate this private data into LLM applications, their usefulness in solving real-world,

¹⁰ This is generally not an option for most businesses due to cost, effort and time required.

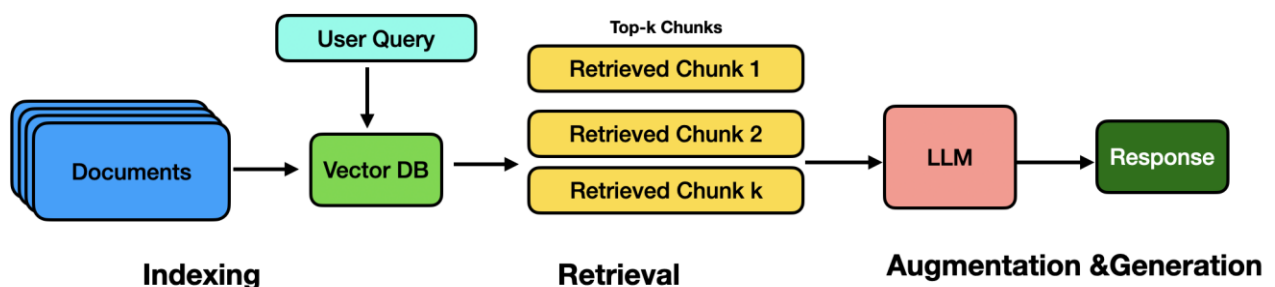
high-stakes business challenges is limited. Retrieval Augmented Generation (RAG) is one of the techniques to effectively use this data.

Unlike traditional methods where an LLM relies solely on its static training corpus, RAG enables real-time integration of external knowledge bases—both public and private. By retrieving relevant context dynamically from these knowledge bases and feeding it into the LLM, RAG ensures that the model can generate accurate, context-aware, and up-to-date responses. This functionality also allows the LLMs to answer based on real time information.

Simple RAG Pipeline

RAG is particularly powerful because it bridges the gap between the generalization capabilities of LLMs and the specificity required for business problems. For example, a customer support application using RAG can provide instant, accurate answers by retrieving information from proprietary FAQs, support tickets, and technical documentation.

Moreover, RAG eliminates the need to fine-tune or retrain LLMs for every new dataset or use case. Instead of feeding proprietary data (in context) directly into the LLM (which may be costly, time-consuming, or even insecure), RAG dynamically retrieves the necessary information at query time, ensuring that sensitive data remains stored securely within the organization's infrastructure.



RAG can be thought of as three step process, (1) Retrieval of relevant information to answer the user query, (2) Augmentation of query and external data, (3) Generation of the final response. However, there is another important step before RAG pipeline can execute on the external data, i.e. indexing of the external data (see Figure 2). Note that this is a simple RAG pipeline with all necessary components.

Indexing

The process involves preprocessing of the raw external data (knowledge base) and indexing of preprocessed data to be stored in a vector index or database. For example, text files or PDF files are split into smaller chunks like paragraphs, sentences, or pages to allow for granular retrieval of the data. This granular chunked data is then converted into vector representations via a pretrained embedding model. These embeddings capture the semantic meaning of the text, enabling context-aware retrieval. Once generated, these embeddings are indexed for fast retrieval and saved in a vector database. Popular tools for this step include FAISS, Pinecone, Weaviate, or Elasticsearch.

Retrieval

We used the term retrieval while talking about indexing but did not elaborate on it. It is in fact carrying the dictionary meaning: *the process of fetching or retrieving relevant information* based on a given query. When a query is presented, it is converted into a vector representation using the same embedding model used during indexing. The query vector is then compared against the stored vectors in the database using similarity metrics such as *cosine similarity* or *Euclidean distance*. The top-k most similar vectors (representing chunks of text) are retrieved, ranked by relevance. These retrieved documents form the context that will be passed to the next stage of the RAG pipeline. Retrieval is the critical bridge between the query and the knowledge base, ensuring that only the most relevant and meaningful information is passed to the generation phase. This step ensures the model can answer domain-specific queries accurately, even when it does not inherently contain the knowledge in its training data.

Augmentation

Augmentation involves enhancing the input to a Large Language Model (LLM) by integrating relevant external data retrieved from a knowledge base in response to the user query. This process includes extracting and summarizing the most pertinent information from the retrieved data to ensure it aligns closely with the context of the original query, enabling the LLM to generate precise and contextually appropriate responses.

Generation

This step uses LLM to generate the response based on the retrieved document from the previous step and the query provided by the user. For example, when a business question involves private data, the answer may lie in the indexed vector database. During the

retrieval process we have retrieved the vector(s) or document(s) that have a strong semantic relationship with the business question.

These retrieved documents contain the information needed to address the query.

Both the query and retrieved documents are now provided to the LLM to generate an enriched and context aware response. This approach enables the integration of private data to answer business-specific questions that would not be possible using only the LLM's training data. By incorporating retrieval results, the model can overcome limitations of static knowledge inherent to pretrained models. The quality of the output depends heavily on:

- The relevance and quality of the retrieved information.
- The capability of the generative model to understand and integrate the retrieved context.

By combining **indexing**, **retrieval**, and **generation**, RAG effectively leverages external knowledge bases and large language models to provide precise, context-aware, and scalable solutions for a variety of applications like customer support, content generation, and domain-specific Q&A.

Advanced RAG

The RAG pipeline shown in Figure 2 is the simplistic version and there has been substantial research in the past year on efficient retrieval of chunks for overall better performance of RAG. There are several techniques to enhance performance, query translation and query routing are common methods used for advanced RAG pipelines.

Query Translation

Let's first try to understand the need for query translation and how their inclusion in the RAG pipeline advances its capabilities and performance. As shown in figure 2 and discussed in the retrieval subsection, "User Query" is an integral part of the pipeline, but not all users are same and hence all users may not write carefully crafted unambiguous query to retrieve the chunks that are most relevant for response. Even a well-crafted, good query may miss important tokens that are necessary for the retrieval of most relevant documents. The semantic similarity search will not be able to retrieve the context to generate good response in the absence of some important tokens. This is where Query Translation comes to rescue. Query translation is the process of (re-)writing efficient prompts using the prompt provided by the user to improve relevance of the retrieved documents. There are many techniques to perform query translation, and some most used ones are:

(a) **Multi-query:** To improve the performance of document retrieval and final responses of the RAG pipeline query enrichment is an important step. Multi-query does this by creating multiple queries based on the original user query. Following is some of the techniques commonly used to rewrite the user query. Note that the rewritten query along with original query can be used to retrieve the documents to obtain diverse response from RAG pipeline.

- a. General Query Rewriting: This approach ([GQR](#)) reduces the noise in user-query by refining the original query. It retains all the relevant information while eliminating the noise from query, thereby enhancing the retrieval precision.
- b. Keyword Rewriting: As the name suggests Keyword rewriting ([KWR](#)) works on the principle of extracting all the keywords in original query, particularly subjects and nouns. By doing this KWR enables search engines to directly address the user query and quickly locate the relevant documents.
- c. Pseudo-Answer Rewriting: Pseudo-Answer Rewriting ([PAR](#)) works on the principle of information expansion. It incorporates the prior information from LLM in the original query from user and uses it for enhanced retrieval. This is done by prompting the LLM to provide pseudo answers to user queries. The underlying hypothesis is that in the embedding space the pseudo answer will be close to the relevant documents. The pseudo answer from LLM may not be factually accurate, but it is expected to be semantically close to the real answer and hence can capture relevant response patterns.
- d. Core Content Extraction: Core Content Extraction (CCE) is a technique to reduce the information present in the user query. In some cases, the user query may contain excessive information, and it is hard for the retriever to extract useful information using the complete query. This will result in discrepancy in retrieved document and user's primary need. An easy solution to this is to discard extra details and extract only the key information from the user query.

(b) **Query Fusion:** In the previous section we saw how multi-query provides different perspectives by rewriting the user query. In the query fusion or commonly called [RAG Fusion](#) multiple queries are formulated using the original user query and integrated with the Reciprocal Ranked Fusion (RRF) with RAG.

Reciprocal Rank Fusion (RRF) is an algorithm designed to merge multiple ranked lists from diverse search methods into a single, cohesive result set. It operates on the principle that items appearing near the top of any individual list are likely to be more relevant. The RRF score for a document is calculated using the formula:

$$\text{RRF Score} = \sum (1 / (k + \text{rank}))$$

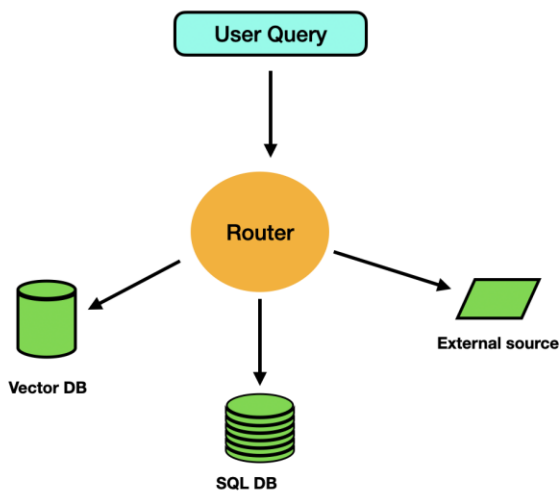
Here, **rank** represents the position of the document in a specific result list, and **k** is a constant (commonly set to 60) that mitigates the impact of lower-ranked documents. By summing these reciprocal values across all lists, RRF effectively boosts documents that consistently rank higher, ensuring that the final aggregated list emphasizes items deemed important by multiple retrieval strategies.

This method retrieves relevant documents for each query, and then re-ranking and merging these documents based on reciprocal scores. This process allows the system to contextualize the original query from various perspectives, thereby providing more accurate and comprehensive responses. Due to multiple retrievals RAG Fusion takes on average 1.7 times w.r.t RAG.

- (c) **Query Decomposition:** Contrary to methods we learn so far this method split the original user query into many smaller parts, referred as subquery. Each of these subqueries are then used to retrieve the relevant document from embedded knowledge base. These retrieved documents along with subqueries are then used for augmenting the summary of the problem which is used by LLM to finally answer the user original query. This method works on the principle of decomposing a complex task into many small tasks that can be easily performed by LLM as these small tasks will find better document matching from the knowledge base.

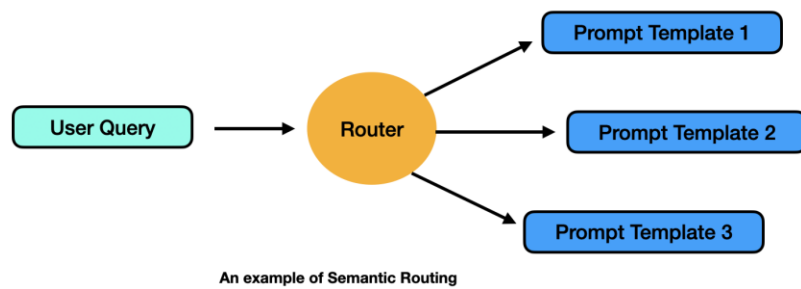
Query Routing

The RAG pipeline we have been discussing has a vector database, but in advanced applications or complex use cases one might have more than one data source. This is a very common situation in big enterprises. The RAG pipeline must route the user query to correct data source ensuring retrieval of the relevant document. For example, if the RAG pipeline has three data-sources; a vector database, a SQL database and data access over API then the query has to be routed to appropriate data source based on the user query. The routing can be thought of as if/else statements, that route the program control to a specified path based on the condition satisfied.



Logical Routing: In this case one can provide LLM information about the data sources we have included in the pipeline and let the LLM decide which data source to use. So, in this case, LLM uses its logical reasoning to make the decision which data source the query should be routed to. This method is particularly useful for creating non-deterministic pipeline, where outcome of one step influence the subsequent steps. By implementing logical routing, the system dynamically decides the most relevant data source or processing route based on the user's query.

Semantic Routing: Semantic routing refers to the process of directing the user query through specific route based on the semantic content. This approach enhances the accuracy of the response by ensuring that user query is handled by the most appropriate module/component in the pipeline. Consider an application where you have multiple options for the prompt template and based on the user queries you want to decide which prompt template should be used for the retrieval of relevant documents. This is also a routing task and all you want to do is route the query to the most relevant prompt template. In this case semantic routing will check semantic similarity and with all the prompt template and route query to the most relevant prompt template.



Toolkit for RAG

The minimal requirement for developing a RAG pipeline is access to LLM, private data, and a vector database to store the indexed dataset. The tools for accessing or setting up a local LLM are the same as discussed in the LLM section. In addition, for RAG pipeline we need tools that can index the data and then save them for future usage.

1. Tools for Data Embedding: OpenAI, Google, Ollama
2. Vector databases: Chroma, Pinecone, Weaviate

Applications of RAG

Most of the use cases discussed in the previous section under application of LLM can benefit from RAG. With the RAG pipeline one can get a better response from LLM, especially when the answer requires usage of private or propriety dataset. The following is a brief description of how they can be advantageous.

1. **Document Summarization and Q/A:** Consider an insurance analyst need to determine whether fire damage is covered under a specific property insurance policy. These documents are generally lengthy, complex and often personalized. Traditionally, this would involve manually reading through the document to locate the relevant sections and this is a time-consuming and error-prone process. By implementing a RAG-based system, the analyst can input a specific query, such as, "Is fire damage covered under property insurance policy number 1234567890?" The system would then retrieve relevant sections from the policy document and generate a concise, accurate response, like, "Yes, fire damage is covered under property insurance policy number 1234567890, as stated in section 4.2." This approach not only reduces the time required to find specific information but also improves accuracy by leveraging both the company's internal data and the language model's understanding capabilities.

2. **Customer Assistance:** The RAG pipeline can also benefit the customer assistance team with searching through the huge number of internal documents and extracting the relevant information to address the question or problem a customer has.
3. **Executive Reporting:** Imagine a case where one needs to prepare a report or summary for the leadership and material for this report is scattered all over emails, SharePoint, CRM, etc. In this scenario, a RAG pipeline connected to these sources can help in preparing and refining the report based on the prompt provided by the user.

Above are just a few use cases in the business where RAG pipeline can be used and add value by saving time and human effort to perform repetitive tasks while keeping the hallucination under control. RAG pipeline can be used for any dataset that is private to the company and willing to extract value out of it using LLM without really shipping the whole dataset to LLM or fine-tuning the LLM.

Multi-Model AI

Remember in the LLMs section we discussed that the input for LLMs should be in text format and for the RAG the documents to be loaded for constructing the knowledge base must be in text format. However, in real life we might have a need to use input in other formats as well, e.g. speech, image, video etc. This is what multi-model AI plays a major role. You are always dealing with an existing multimodal system from a very long time, human being.

Human perception is inherently multimodal, it integrates inputs from various senses; vision, hearing, touch, taste, and smell; to construct a cohesive understanding of the environment. This natural ability to process and synthesize diverse sensory information enables humans to interact with the world in a significant and interactive manner. The brain serves as the central organ that combines all these sensory modalities, enabling humans to interact with the world. Emulating this capability in artificial intelligence (AI) systems has led to the development of multimodal AI¹¹, which aims to process and integrate multiple types of data inputs, such as text, images, audio, and video, to achieve a more comprehensive understanding and generate more accurate outputs.

¹¹ interchangeably referred as multimodal system

Modality and Multimodality

In the context of AI and machine learning, a **modality** refers to a specific type of data or input that a system can process. Common modalities include textual data, visual data (images and videos), auditory data (speech and sounds), and sensory data (such as touch or temperature readings). Each modality provides unique information and requires specialized processing techniques. **Multimodality** involves the integration and simultaneous processing of multiple such modalities within a single AI system, enabling it to leverage the complementary nature of different data types to enhance understanding and performance. For example, an image of a beautiful landscape has some information and if one describes this image in the words (natural language) it contains information about the scene. However, when the textual description is fused with the information in image, it provides complimentary information and hence becomes more informative and relevant.

Examples and Types of Multimodalities

Examples of modalities include:

- **Textual Data:** Written language, documents, and textual records.
- **Visual Data:** Images, videos, and visual representations.
- **Auditory Data:** Speech recordings, music, and environmental sounds.
- **Sensory Data:** Inputs from sensors measuring temperature, pressure, or motion.

Principle of Modality

In **multimodal AI** there are three fundamental principles that contribute to constructing a cohesive and comprehensive understanding of data sources: Heterogeneity, Connections and Interactions.

Heterogeneity

Heterogeneity refers to the inherent diversity in the qualities, structures, and representations of different data modalities. For instance, textual data is sequential and symbolic, conveying information through language, whereas visual data, such as images or videos, is spatial and often continuous, capturing scenes or objects in a manner that words alone cannot. This diversity extends to other modalities like audio, which encompasses temporal patterns and frequencies, and sensory data from devices measuring various physical parameters. Each modality offers a unique perspective, and their combined analysis enables AI systems to capture a more holistic view of the information landscape.

Connections

Connections pertain to the complementary relationships between different modalities. While each modality provides distinct information, there are often underlying associations that link them. For example, a textual description of an event and a photograph capturing the same moment are connected through the shared context they represent. Identifying and leveraging these connections allow multimodal AI systems to align and integrate data effectively, enriching contextual understanding and enabling more accurate interpretations.

Interactions

Interactions involve the dynamic processes through which information from multiple modalities influences and enhances the interpretation of each other. In multimodal AI, interactions are modeled to capture how different data types of interplay, leading to emergent insights that would not be apparent when considering each modality in isolation. For instance, in sentiment analysis, combining textual data with vocal tone and facial expressions provides a deeper understanding of emotional states than any single modality could offer. Modeling these interactions is crucial for developing AI systems that can reason across modalities, effectively mimicking the integrative nature of human perception.

By addressing the challenges posed by heterogeneity, establishing meaningful connections, and effectively modeling interactions, multimodal AI systems can synthesize diverse information sources into a unified and coherent representation. This integrative approach enhances the system's ability to perform complex tasks, such as understanding nuanced human communication, interpreting multimedia content, and making informed decisions based on a rich tapestry of data inputs.

Model Availability and Toolkit

Although the GenAI models started with a focus on the language tasks, the research in multimodality has evolved rapidly in the last few years. There exists both closed source and open-sourced models that support multimodality; especially text and image. Combination of just text and image can have many use cases in the business to address complex challenges involving image with text. GPT from Open AI and Gemini from Google offers access to their multimodal AI via API. However, the recent release of Llama series from Meta has capabilities to process multimodal data and it is open sourced.

Since the multimodal versions are merely an extension of the Language models, they are accessible via the same set of tools as Language models. The toolkit to leverage and these models remains the same.

Use Case for Multi-modal AI

1. **Document Analysis:** Companies receive thousands of physical documents every year, including tax forms, contracts, legal documents, and compliance records. The processing of these documents poses a big challenge to the business as this requires manual effort, is time-consuming and the process is highly error prone. By using multimodal AI model that can process vision and language data companies can automate the extraction and interpretation of physical documents. The AI model can extract the information from scanned images of documents, handwritten form or printed reports and extract key information.
2. **Visual Question-Answering:** Another interesting application of multimodal AI is understanding image data. One can leverage AI by asking questions based on the image data and extract important business insight. This image can be a graphics, a bar chart of some other business-related document with image and text. For e.g. an analyst can provide a chart and ask questions to extract insight from the chart. Instead of spending time manually analyzing trends, they can upload the image to the AI model and ask, *"What are the key takeaways from this chart?"* The model identifies the patterns, and it could respond like "The company has shown consistent growth in past four quarters".

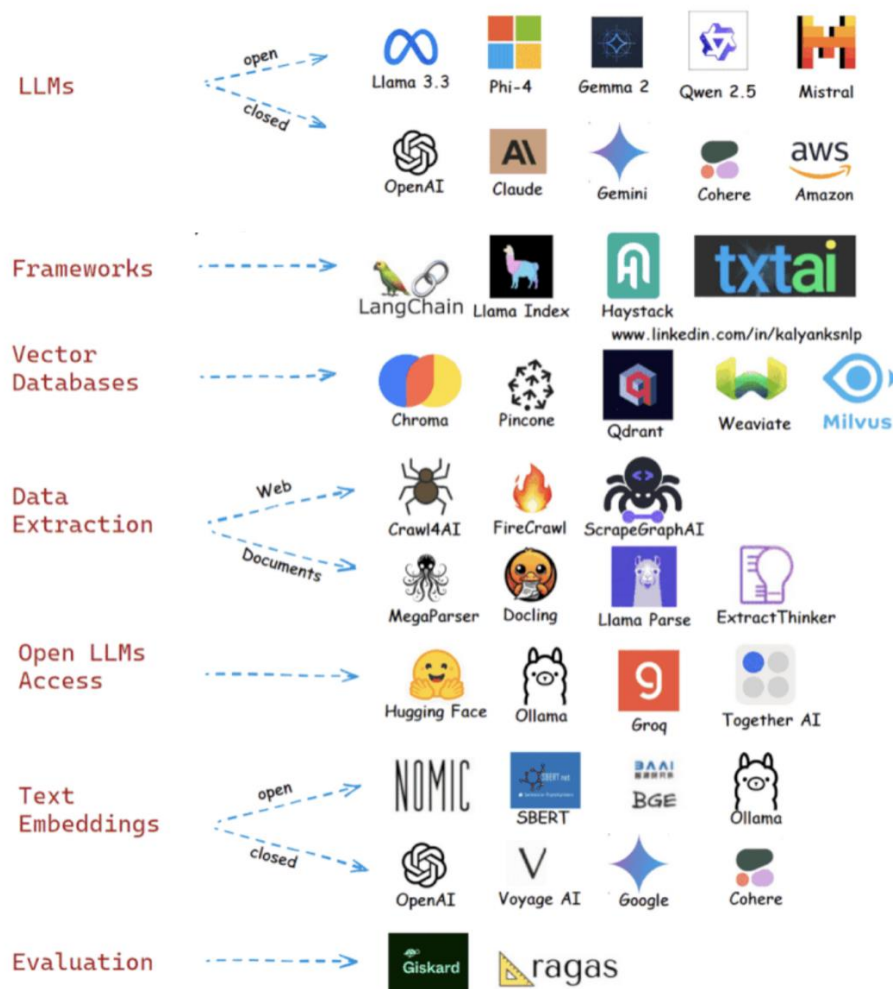
Another e.g. is the user asking question based on chart and a user manual. In this case, a user asks for an explanation of the image but the image itself is not sufficient to provide any explanation. Informative explanation requires processing of a user manual that can be provided as an additional input to the multimodal. The AI is now equipped with all the information it needs to explain the image. This will generally require a human to go through the lengthy and technical manual to extract the relevant information.

Conclusions

In this report, we explored several key concepts, the tools available to leverage them, and their applications in a business setting. The use cases discussed serve as illustrative examples, aimed at sparking ideas on the potential of LLMs and multimodal AI rather than providing an exhaustive list. The true value of these technologies lies in their adaptability, enabling businesses to tailor them to their specific challenges and operational needs.

Beyond conceptual discussions, we also briefly examined various tools that can be employed for different applications when working with LLMs. To provide a structured

overview, the following section presents a categorized summary of these tools, highlighting their functionalities and use cases.



References

[1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. arXiv preprint arXiv:1706.03762.

[2] Hopfield, J. J. (1982). Neural networks and physical systems with emergent collective computational abilities. Proceedings of the National Academy of Sciences, 79(8), 2554–2558. <https://doi.org/10.1073/pnas.79.8.2554>

- [3] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536. <https://doi.org/10.1038/323533a0>
- [4] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- [5] Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to Sequence Learning with Neural Networks. *Advances in Neural Information Processing Systems*, 27, 3104–3112. <https://arxiv.org/abs/1409.3215>
- [6] Luong, M.-T., Pham, H., & Manning, C. D. (2015). Effective Approaches to Attention-based Neural Machine Translation. *arXiv preprint arXiv:1508.04025*.
- [7] Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. *arXiv preprint arXiv:1409.0473*. <https://arxiv.org/abs/1409.0473>
- [8] Luong, M.-T., Pham, H., & Manning, C. D. (2015). Effective Approaches to Attention-based Neural Machine Translation. *arXiv preprint arXiv:1508.04025*. <https://arxiv.org/abs/1508.04025>
- [9] Gehring, J., Auli, M., Grangier, D., Yarats, D., & Dauphin, Y. N. (2017). Convolutional Sequence to Sequence Learning. *arXiv preprint arXiv:1705.03122*. <https://arxiv.org/abs/1705.03122>
- [10] Kalchbrenner, N., Espeholt, L., Simonyan, K., van den Oord, A., Graves, A., & Kavukcuoglu, K. (2016). Neural Machine Translation in Linear Time. *arXiv preprint arXiv:1610.03035*. <https://arxiv.org/abs/1610.03035>